

AI Usage Log and Reflection Report

Student Name: Manan Shori

Course: SE ZG503 – Full Stack Application Development

Project: secure-password-wallet (Microservices-based Password Manager)

Approach: Option A – Build from scratch with AI assistance

1. AI Tools and Methodology

For this assignment, I utilized the **Antigravity AI agent** (powered by **Gemini 3 Flash**) as my primary pair-programming collaborator. The AI was used throughout the software development life cycle (SDLC), specifically for:

- **System Scaffolding:** Generating the folder structure for a microservice architecture.
- **Boilerplate Generation:** Writing initial Express.js routes and React component shells.
- **Cryptographic Logic:** Implementing the AES-256-GCM encryption flow in the Vault Service.
- **Documentation:** Drafting the initial API endpoints and system walkthrough.

2. Example Prompts Used

The following prompts were instrumental in moving from the problem statement to a functioning prototype:

Phase	Prompt Example
Scaffolding	"Scaffold a secure password wallet using a microservice architecture with React, Node.js,

	and MongoDB. Separate Auth and Vault into different services."
Logic	"Implement AES-256-GCM encryption in the Vault service to secure user passwords before saving them to MongoDB."
Debugging	"I am seeing a 'Failed to register' error on localhost:3000. The backend started but MongoDB appears inactive. How do I fix the connection lifecycle?"
DevOps	"GitHub returned an 'Invalid username or token' error during push. Help me configure a Personal Access Token (PAT) for manual terminal authentication."

3. Parts Generated by AI vs. Manually Coded

A significant portion of the logic was generated by AI, but the "operational reality" required extensive manual intervention.

- **Generated by AI:**
 - React component shells (Login, Register, Dashboard).
 - Standard REST API route boilerplate for CRUD operations.
 - Initial MongoDB schemas for Users and Credentials.
- **Manually Developed/Resolved:**
 - **Environment Setup:** Manual installation of Homebrew and MongoDB Community Edition on macOS.
 - **Configuration:** Updating system PATH variables to make mongod and mongosh accessible.
 - **Database Connectivity:** Troubleshooting the service startup order to ensure the

- database was listening before the Node.js services attempted connection.
- **Authentication:** Manually managing GitHub Personal Access Tokens (PAT) after AI-driven push attempts failed due to token revocation.

4. Reflection: Learning Outcomes and Challenges

Did AI Help or Hinder?

AI significantly helped in **velocity**. It eliminated the "blank page" problem by providing a functioning microservice skeleton in minutes. However, it occasionally hindered deep architectural understanding initially, as it handled the complex service-to-service communication "magically" in the background.

Issues Encountered with AI Output

The most significant issue was the **"Environmental Gap"**. The AI generated code assuming a "perfect" local environment where MongoDB was already running and Git was pre-authenticated. When these assumptions failed on my local MacBook Air, the AI-generated error messages were sometimes generic, requiring me to step out of the AI chat and into the system terminal for manual debugging.

Lessons from Debugging AI-Generated Code

I learned that **Integration is more difficult than Implementation**. While the AI can write a perfect POST route, it cannot ensure that my local machine has the correct database permissions or that GitHub's security bots won't revoke a token shared in a log file. Debugging the "Failed to register" error taught me the importance of **service synchronization** and health checks in a microservice environment.

5. Comparison: Manual vs. AI Workflow

Metric	Manual Workflow	AI-Assisted Workflow
Speed	Slow; required looking up documentation for every	Very High; generated 80% of boilerplate instantly.

	library.	
Accuracy	High; manual control over every line of code.	Variable; required frequent "Sanity Checks".
Debugging	Clearer; I knew exactly what I wrote.	Complex; had to trace AI logic to find where connection strings failed.

Conclusion: The AI-assisted workflow is superior for rapid prototyping, but the role of the developer has shifted from "Writer" to "Architect and Debugger." For a production-grade system like **secure-password-wallet**, human oversight remains the final line of defense for security and environment-specific integration.